

CORTEX v3.0

Clinical Risk Assessment System

AI-Powered Clinical Deterioration Detection
& Patient Risk Monitoring Platform

Trained on MIMIC-IV Clinical Database (1.41M+ patients)
Deployed on Google Cloud Platform

Model Accuracy: 99.98% | High-Risk Recall: 100%
Inference Latency: < 1ms (P95)

Comprehensive Project Documentation
Version 3.0 | February 2026

Table of Contents

- 1. Executive Summary
- 2. Problem Statement & Motivation
- 3. System Architecture
- 4. Technology Stack
- 5. Machine Learning Model
 - 5.1 Training Data & Sources
 - 5.2 Feature Engineering (44 Features)
 - 5.3 Model Training & Selection
 - 5.4 Performance Metrics
 - 5.5 Safety Override System
 - 5.6 Threshold Optimization
 - 5.7 Clinical Test Cases
- 6. Backend Architecture (cortex-agent)
 - 6.1 FastAPI Server & Endpoints
 - 6.2 Cortex Unified Agent
 - 6.3 ML Predictor Module
 - 6.4 AI Chat Client (LLM Integration)
 - 6.5 Firebase Database Layer
- 7. Frontend Architecture (React Dashboard)
 - 7.1 Component Structure
 - 7.2 Dashboard Tabs Overview
 - 7.3 Authentication System
 - 7.4 UI/UX Design
- 8. Security & Compliance
- 9. Deployment & Infrastructure
- 10. API Documentation
- 11. Project File Structure
- 12. Performance Benchmarks
- 13. Future Roadmap

1. Executive Summary

CORTEX v3.0 is a production-ready, AI-powered clinical risk assessment system designed to predict patient deterioration risk in real-time. It combines a high-accuracy machine learning model (XGBoost ensemble) with an intuitive React dashboard, enabling healthcare professionals and patients to monitor health status with clinical-grade accuracy.

The system is trained on the MIMIC-IV Clinical Database, comprising over 1.41 million cleaned patient records from ICU admissions. The ML model achieves 99.98% overall accuracy with 100% high-risk recall, meaning it has never missed a single critical patient in testing across 26,351 high-risk cases.

The platform features a full-stack architecture: a Python/FastAPI backend deployed on Google Cloud Run, a React 19 SPA frontend with Firebase Authentication and Firestore for real-time data, and an AI health companion powered by LLM integration (Llama 3.3 70B via OpenRouter) for conversational health guidance, document analysis, and assessment explanations.

Key Highlights

- 99.98% Overall Accuracy with 100% High-Risk Recall (0 missed critical patients)
- Sub-millisecond inference (0.39ms P95) - 550,000+ predictions/second throughput
- 44 engineered clinical features from 7 core vital signs
- Safety override system for critical thresholds (SpO2, HR, BP, Temp)
- Real-time React dashboard with 6 interactive tabs
- AI health companion with conversational chat, document OCR, and assessment explanations
- Firebase Auth + Firestore with user-scoped security rules
- Dockerized and deployed on Google Cloud Run (asia-south1)
- Trained on 1,410,937 real patient records (no synthetic data)

2. Problem Statement & Motivation

Clinical deterioration in hospitalized patients is a leading cause of preventable deaths. Studies show that early warning signs are often present 6-24 hours before cardiac arrest or ICU transfer, but are frequently missed due to information overload, staffing shortages, and reliance on manual monitoring.

Traditional Early Warning Score (EWS) systems like NEWS and MEWS use simple threshold-based rules that fail to capture complex, non-linear interactions between vital signs. They suffer from high false alarm rates (leading to "alarm fatigue") and inadequate sensitivity for truly critical patients.

CORTEX addresses these challenges by applying advanced machine learning to vital sign analysis, providing:

- Automated risk stratification into Low / Medium / High categories
- Explainable predictions with feature importance rankings
- Safety override rules for critical thresholds that bypass the ML model
- Natural language explanations of results for patient comprehension
- Real-time monitoring through an accessible web dashboard
- AI-powered conversational health guidance for patient empowerment

3. System Architecture

CORTEX follows a modern three-tier architecture separating the frontend presentation layer, the backend API/ML inference layer, and the cloud data persistence layer.

Architecture Overview

The system consists of three main components that communicate via REST APIs over HTTPS:

1. Frontend (React 19 SPA)

- Built with React 19.2.0, Vite 7.3.1, and Tailwind CSS 4.2.0
- Single Page Application with client-side routing (React Router 7.13)
- Communicates with Firebase for auth/data and Cloud Run for ML predictions
- Deployed on Firebase Hosting / Vercel

2. Backend (FastAPI on Cloud Run)

- Python 3.11 FastAPI server containerized with Docker
- Hosts the trained XGBoost ML model for risk predictions
- Integrates LLM (Llama 3.3 70B via OpenRouter) for chat and explanations
- Connects to Firebase Firestore for persistent data storage
- Deployed on Google Cloud Run (asia-south1 / us-central1)

3. Data Layer (Firebase + GCP)

- Firebase Authentication for user management (email/password + OTP)
- Cloud Firestore (NoSQL) for user data, assessments, documents, reminders
- Security rules enforce user-scoped access (users can only access their own data)
- Google Cloud Storage for model artifact storage

Data Flow

1. User logs in via Firebase Auth (email/password with OTP verification)
2. Dashboard loads user profile and assessment history from Firestore
3. User inputs vital signs on the Assessment tab
4. Frontend sends vitals to the Cloud Run backend via POST /assessment
5. Backend runs safety overrides, then ML inference (XGBoost), then LLM explanation
6. Results are returned to frontend and saved to Firestore
7. Dashboard updates in real-time with risk scores, analytics, and recommendations

4. Technology Stack

Frontend Technologies

Technology	Version	Purpose
React	19.2.0	UI component library
Vite	7.3.1	Build tool & dev server
React Router	7.13.0	Client-side SPA routing
Tailwind CSS	4.2.0	Utility-first CSS framework
Lucide React	0.575.0	Icon library
Firebase SDK	12.9.0	Auth + Firestore client
Axios	1.13.6	HTTP client for API calls
React Markdown	10.1.0	Markdown rendering in chat
EmailJS	4.4.1	Email OTP delivery

Backend Technologies

Technology	Version	Purpose
Python	3.11	Runtime environment
FastAPI	0.134.0	Async web framework
Uvicorn	0.41.0	ASGI server
scikit-learn	1.8.0	ML library
XGBoost	latest	Gradient boosting classifier
NumPy	2.4.2	Numerical computing
Pandas	3.0.1	Data manipulation
Firebase Admin	7.2.0	Server-side Firestore
OpenAI SDK	2.24.0	LLM client (OpenRouter)
PyPDF2	3.0.1	PDF document processing
python-docx	1.2.0	Word document processing
Pillow	12.1.1	Image processing

Infrastructure & DevOps

Cloud Provider:	Google Cloud Platform (GCP)
ML Hosting:	Google Cloud Run (containerized serverless)
Database:	Cloud Firestore (NoSQL, real-time)
Authentication:	Firebase Authentication
Frontend Hosting:	Firebase Hosting / Vercel
Container Registry:	Google Container Registry (GCR)
Region:	asia-south1 (Mumbai) / us-central1
Project ID:	cortex-12feb

5. Machine Learning Model

5.1 Training Data & Sources

The CORTEX model is trained exclusively on real patient data from multiple clinical databases. No synthetic data (SMOTE or oversampling) is used -- the model learns purely from real-world clinical observations.

Dataset	Records	Description
Sepsis Challenge	1,552,210	PhysioNet Sepsis Challenge data
CHARTEVENTS (MIMIC)	758,355	MIMIC-IV charted vitals
Stroke Data	5,110	Stroke patient records
MIMIC-IV ED	1,038	Emergency dept vitals

Total Raw Records:	1,568,014
After Cleaning:	1,410,937 (90.0% retention)
Training Set (80%):	1,128,749 records
Test Set (20%):	282,188 records

Class Distribution

Risk Level	Count	Percentage	Weight
Low Risk	811,293	57.5%	1x
Medium Risk	467,887	33.2%	2x
High Risk	131,757	9.3%	8x

The labeling strategy in v3.0 was fixed to be based purely on observable vital signs, eliminating information leakage from outcome variables (sepsis labels, disposition codes) that existed in v2.0. This ensures the model can learn all high-risk patterns from features it actually has access to.

5.2 Feature Engineering (44 Features)

CORTEX engineers 44 features from 7 core vital signs, demographics, and medical history. Features are organized into 12 categories:

Category	Features
Basic Vitals (7)	hr, spo2, sbp, dbp, rr, temp, map
Demographics (3)	age, gender, age_group
Medical History (3)	diabetes, hypertension, heart_disease
Derived Vitals (3)	pulse_pressure, shock_index, mews_score
Abnormal Flags (6)	hr/spo2/bp/temp/rr_abnormal, total_abnormal_vitals
Composite Scores (4)	vital_instability, cardio_risk, respiratory_risk, critical_score
Interactions (4)	hr_spo2_ratio, symptom_count, pp_ratio, dbp_sbp_ratio
Squared/Deficit (3)	hr_squared, spo2_deficit, spo2_deficit_sq
Age Features (1)	age_risk
Deviations (3)	hr_deviation, sbp_deviation, rr_deviation
Critical Flags (5)	critical_spo2/hr/bp/rr/temp_flag
Emergency (2)	multi_critical_flag, emergency_score

Top 10 Feature Importance

#	Feature	Importance	Category
1	emergency_score	33.46%	Emergency
2	critical_rr_flag	20.28%	Critical Flags
3	critical_score	14.29%	Composite Scores
4	total_abnormal_vitals	13.33%	Abnormal Flags
5	hr_abnormal	5.39%	Abnormal Flags
6	sbp_deviation	5.10%	Deviations
7	critical_bp_flag	1.95%	Critical Flags
8	dbp	1.52%	Basic Vitals
9	temp_abnormal	0.64%	Abnormal Flags
10	mews_score	0.58%	Derived Vitals

5.3 Model Training & Selection

Multiple models were trained and compared. The selection criteria prioritized high-risk recall (the most critical metric for patient safety), followed by overall accuracy, overfitting gap, and inference speed.

Training Configuration

Train/Test Split:	80% / 20% (stratified by risk level)
Feature Scaling:	StandardScaler
Class Weights:	{Low: 1, Medium: 2, High: 8}
Cross-Validation:	10-fold stratified
Hyperparameters:	n_estimators=400, max_depth=8, lr=0.1
Random State:	42 (reproducible)
Training Time:	17 minutes 15 seconds

Model Comparison

Model	Test Acc	HR Recall	Train Acc	Time
XGBoost	99.98%	100.00%	100.00%	57.2s
Random Forest	100.00%	99.99%	100.00%	238.7s
RF Heavy-Weight	100.00%	99.99%	100.00%	281.7s
Extra Trees	99.75%	99.98%	99.89%	262.2s

Selected Model: XGBoost -- highest high-risk recall with fastest inference time.

5.4 Performance Metrics

Overall Metrics

Overall Accuracy:	99.98%
Macro F1-Score:	0.9999
Weighted F1-Score:	0.9998
Cohen's Kappa:	0.9997
Matthews Correlation Coefficient	0.9997
ROC-AUC (macro):	1.0000
ROC-AUC (weighted):	1.0000

Per-Class Metrics

Class	Precision	Recall	F1	Support
Low Risk	99.97%	100.00%	0.9999	162,259
Medium Risk	100.00%	99.95%	0.9997	93,578
High Risk	100.00%	100.00%	1.0000	26,351

Confusion Matrix

Actual \ Pred	Low	Medium	High
---------------	-----	--------	------

Low Risk	162,256	3	0
Medium Risk	45	93,533	0
High Risk	0	0	26,351

Total errors: 48 / 282,188 (0.02%). Zero high-risk patients were missed.

Overfitting Analysis

Train Accuracy: 100.00%

Test Accuracy: 99.98%

Gap: 0.02% (well within 3% target)

CV Mean: 99.98% +/- 0.01% (10-fold)

5.5 Safety Override System

The safety override system acts as a hard-coded clinical safety net that bypasses the ML model entirely when vital signs breach critical thresholds. This ensures that dangerously abnormal vitals are ALWAYS classified as High Risk, regardless of what the ML model predicts.

Vital Sign	Threshold	Clinical Significance
SpO2	< 90%	Severe hypoxemia
Heart Rate	> 150 or < 40 bpm	Critical tachycardia/bradycardia
Systolic BP	> 180 mmHg	Hypertensive crisis
Systolic BP	< 90 mmHg	Hypotensive shock
Diastolic BP	> 110 mmHg	Hypertensive crisis
Diastolic BP	< 60 mmHg	Hypotensive crisis
Temperature	> 103 F	High fever / Hyperthermia
Temperature	< 95 F	Hypothermia

When triggered, the system returns `risk_category="High"`, `confidence=1.0`, `safety_override=true`, and an `override_reason` describing which threshold was breached.

5.6 Threshold Optimization

The optimal probability threshold for high-risk classification was determined through systematic evaluation. A threshold of 0.15 was selected (vs default 0.50) to maximize sensitivity for critical patients.

Optimal Threshold: 0.15

Default Threshold: 0.50

At the optimal threshold: High-Risk Recall = 100.00%, High-Risk Precision = 100.00%, Overall Accuracy = 99.98%. The model maintains perfect performance across all tested thresholds.

5.7 Clinical Test Cases

#	Test Case	Predicted	Expected	Result
1	Normal healthy patient	Low	Low	PASS
2	Moderate risk patient	Medium	Medium	PASS
3	Critical patient	High	High	PASS
4	Safety: SpO2=85%	High	High	PASS
5	Safety: HR=160	High	High	PASS
6	Safety: SBP=85	High	High	PASS
7	Borderline case	Low	Low/Med	PASS
8	Elderly w/ comorbidities	Low	Medium	FAIL
9	Isolated hypertension	High	Med/High	PASS

Results: 8/9 passed. All 3 safety override cases passed. The single failure (elderly with comorbidities) is a known edge case where age + comorbidities alone may not trigger elevated risk without accompanying vital sign abnormalities.

6. Backend Architecture (cortex-agent)

The backend is a Python FastAPI application that serves as the intelligence layer of CORTEX. It orchestrates ML predictions, LLM-powered chat, document analysis, and persistent data storage.

6.1 FastAPI Server & Endpoints

The server exposes the following REST API endpoints:

Method	Endpoint	Description
POST	/chat	Conversational health queries via LLM
POST	/assessment	ML risk prediction from vital signs
POST	/upload-document	Medical document OCR and analysis
GET	/documents	Retrieve user documents
DELETE	/documents/{id}	Delete a document
POST	/documents/share	Mark document as shared
GET	/dashboard	Get user dashboard data
POST	/reminder	Create health/medication reminder
GET	/history	Get conversation history
DELETE	/history	Clear conversation context

6.2 Cortex Unified Agent

The CortexUnifiedAgent class (cortex_agent.py) is the central orchestrator that routes requests between the ML predictor, LLM client, and database layer. It provides four main capabilities:

- handle_chat() - Routes conversational queries to the LLM with assessment context
- handle_assessment() - Runs ML prediction + LLM explanation + database save
- handle_document_upload() - Sends documents to LLM for OCR/analysis + saves results
- handle_create_reminder() - Creates health reminders in Firestore

6.3 ML Predictor Module

The CortexMLPredictor class (ml_predictor.py) handles the complete ML inference pipeline:

- Safety Override Check - Hard-coded clinical thresholds checked before ML inference
- Feature Engineering - Expands 6 core vitals into the full feature vector
- StandardScaler Transform - Normalizes features using the trained scaler
- XGBoost Prediction - Generates class probabilities [Low, Medium, High]
- Fallback Heuristics - Rule-based prediction if model files are unavailable

6.4 AI Chat Client (LLM Integration)

The CortexGeminiClient class (gemini_client.py) provides LLM-powered capabilities via OpenRouter:

Primary Model: Llama 3.3 70B Instruct (free tier)

Fallback Model: OpenRouter Auto-Router (free)
API Provider: OpenRouter (OpenAI-compatible API)

The LLM is used for three functions:

- generate_chat_response() - Free-form health Q&A with conversation history
- explain_assessment() - Plain-language explanation of ML risk predictions
- analyze_document() - Medical document OCR, categorization, and key findings extraction

The system prompt defines the AI persona as "Cortex Health Companion" -- warm, empathetic, patient-facing, with strict safety rules for emergency detection and medical disclaimers.

6.5 Firebase Database Layer

The Database class (database.py) provides a Firestore persistence layer with mock fallback. Data is organized under per-user document collections:

- users/{userId}/history - Conversational memory (chat messages)
- users/{userId}/assessments - Risk assessment results with vitals and predictions
- users/{userId}/documents - Uploaded medical document analyses
- users/{userId}/reminders - Health and medication reminders

7. Frontend Architecture (React Dashboard)

7.1 Component Structure

The frontend is a React 19 SPA built with Vite 7 and styled with Tailwind CSS 4. It follows a component-based architecture with the main Dashboard component routing between six tab views.

Core Components

- App.jsx - Root component with React Router (/, /login, /signup, /dashboard)
- Dashboard.jsx - Main dashboard container with tab routing and data management
- AuthContext.jsx - Authentication provider (signup, login, logout, OTP, Firestore sync)
- Navbar.jsx, Hero.jsx, Features.jsx, etc. - Landing page components

Dashboard Tab Components

- HomeTab.jsx - Patient overview, Cortex AI score, health summary, care timeline
- HealthChatTab.jsx - AI health companion chat interface
- VitalAssessmentTab.jsx - Vital sign input form with ML risk prediction
- DocumentsTab.jsx - Clinical document management
- DatabaseTab.jsx - Assessment history and biometric logs
- AnalyticsInsights.jsx + BrainMapping.jsx + VitalsGrid.jsx + NextSteps.jsx - Analytics

7.2 Dashboard Tabs Overview

Home Tab

The main dashboard view showing a personalized greeting, Cortex AI health score (0-100), health summary cards (appointments, medications, risk level, timeline), latest assessment results with probability breakdown, care timeline, and user profile with action buttons.

Health Chat Tab

An AI-powered conversational health assistant. Users can ask any health question and receive contextual, empathetic responses. The chat maintains conversation history and can reference the user's latest assessment data. Responses are rendered in Markdown format.

Assessment Tab

A vital sign input form where users enter Heart Rate, SpO2, Blood Pressure, Temperature, and Respiratory Rate. On submission, the data is sent to the ML backend for real-time risk prediction. Results show risk category, confidence, and probability breakdown.

Documents Tab

A clinical document management system supporting upload and AI-powered analysis of medical documents (lab reports, imaging, clinical notes). Documents are categorized, summarized, and key findings are extracted using LLM analysis.

Database Tab

Displays assessment history with timestamps, biometric logs, auto-scan status, and system statistics. Users can view

and manage their historical health data.

Analytics Tab

Advanced health analytics including: Brain Mapping (SHAP-inspired feature importance visualization), Personal Risk Gauge, Interactive Vitals Grid with historical trends, and a personalized Next Steps action plan with recommendations.

7.3 Authentication System

CORTEX uses Firebase Authentication with a custom OTP verification layer:

- Email/Password Registration with display name
- Email OTP Verification via Firebase Cloud Functions + EmailJS
- Login with "Keep me signed in" option (browser local vs session persistence)
- Password Reset via Firebase sendPasswordResetEmail
- Auth State Caching in localStorage for instant UI restore on page reload
- Real-time Firestore listener (onSnapshot) syncs user data and assessments
- Protected route guards redirect unauthenticated users to the landing page

7.4 UI/UX Design

The dashboard features a premium, modern design language:

- Clean white background with subtle ambient mesh gradient backgrounds
- Glass morphism panels with soft shadows and rounded corners (32px border radius)
- Inter font family for clean, readable typography
- Consistent color palette: emerald/teal (#00DAAA, #109981) for health indicators
- Animated skeleton loaders during data fetching for smooth UX
- Responsive mobile-first layout using Tailwind CSS grid and flexbox
- Lucide React icons throughout the interface

8. Security & Compliance

Authentication & Authorization

- Firebase Authentication with email/password (industry-standard)
- OTP email verification prevents unauthorized account creation
- Session persistence options (local storage vs session-only)
- Auto-logout on token expiration

Firestore Security Rules

User-scoped access control ensures data isolation:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /otpVerifications/{docId} {
      allow read, write: if false; // Cloud Functions only
    }
    match /users/{userId}/{document=**} {
      allow read, write: if request.auth != null
        && request.auth.uid == userId;
    }
  }
}
```

Data Privacy

- All data scoped to individual users (no cross-user access)
- No PHI stored in frontend localStorage (only auth state)
- All API communication over HTTPS (TLS 1.3)
- CORS middleware enabled on backend
- OTP verification records are Cloud Functions-only (no client access)
- HIPAA-eligible infrastructure (Firebase + GCP)

9. Deployment & Infrastructure

Backend Deployment (Google Cloud Run)

The backend is containerized using Docker and deployed to Google Cloud Run:

Dockerfile

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV PORT=8080
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE $PORT
CMD ["sh", "-c", "uvicorn api.main:app --host 0.0.0.0 --port $PORT"]
```

Cloud Run Configuration

Region:	asia-south1 (Mumbai)
Memory:	2 GB
CPU:	2 vCPUs
Timeout:	300 seconds
Max Instances:	10
Min Instances:	0 (scales to zero)
Authentication:	Allow unauthenticated (public API)

Frontend Deployment

The frontend can be deployed to Firebase Hosting or Vercel:

Firebase Hosting

- Build: `npm run build` (Vite produces optimized `dist/` folder)
- Deploy: `firebase deploy --only hosting`
- SPA rewrites configured for client-side routing
- Cache headers set to no-cache for HTML/JS/CSS

Vercel

- `vercel.json` configured with SPA rewrites
- Environment variables set in Vercel dashboard

10. API Documentation

Base URLs

Production: `https://cortex-agent-472595500035.us-central1.run.app`
Local Dev: `http://localhost:8000`

POST /chat

Handles free-form conversational health queries via LLM.

Request Body

```
{
  "user_id": "string",
  "message": "string",
  "latest_assessment": { ... }, // optional
  "chat_history": [ ... ]      // optional
}
```

Response

```
{ "status": "success", "response": "AI response text" }
```

POST /assessment

Processes vital signs through the ML model and returns risk prediction with explanation.

Request Body

```
{
  "user_id": "string",
  "vitals": {
    "hr": 95, "spo2": 97, "sys_bp": 110,
    "dia_bp": 70, "rr": 18, "temp": 98.6
  }
}
```

Response

```
{
  "status": "success",
  "data": {
    "prediction": {
      "risk_category": "Low",
      "risk_score": 1,
      "probabilities": { "Low": 85.0, "Medium": 13.0, "High": 2.0 },
      "confidence": 0.94,
      "safety_override": false
    },
  },
}
```

```
    "explanation": "AI-generated plain language explanation"
  }
}
```

POST /upload-document

Uploads and analyzes medical documents (multipart form data).

Content-Type: multipart/form-data

Fields: user_id (string), file (binary)

11. Project File Structure

```

Cortex/
|-- README.md                # Project documentation
|-- train_cortex_v3.py        # Model training script (v3.0)
|-- evaluate_cortex_v3.py     # Model evaluation script
|-- model_production.pkl      # Trained XGBoost model (2.8 MB)
|-- scaler_production.pkl     # StandardScaler artifact
|-- feature_names_production.pkl # 44 feature names
|-- threshold_config.json     # Optimal threshold config
|-- model_metadata.json       # Full model metadata
|-- training_report_production.txt # Training report
|
|-- cortex-agent/             # Backend (Python/FastAPI)
|   |-- api/
|   |   |-- main.py           # FastAPI server + endpoints
|   |-- agent/
|   |   |-- cortex_agent.py   # Unified agent orchestrator
|   |   |-- ml_predictor.py   # ML inference wrapper
|   |   |-- gemini_client.py  # LLM client (OpenRouter)
|   |   |-- database.py       # Firestore persistence
|   |-- models/               # Model artifacts
|   |-- requirements.txt       # Python dependencies
|   |-- Dockerfile            # Container definition
|
|-- Landing Page/             # Frontend (React/Vite)
|   |-- src/
|   |   |-- components/
|   |   |   |-- Dashboard.jsx # Main dashboard
|   |   |   |-- Hero.jsx, Navbar.jsx, etc.
|   |   |   |-- dashboard/
|   |   |       |-- HomeTab.jsx
|   |   |       |-- HealthChatTab.jsx
|   |   |       |-- VitalAssessmentTab.jsx
|   |   |       |-- DocumentsTab.jsx
|   |   |       |-- DatabaseTab.jsx
|   |   |       |-- AnalyticsInsights.jsx
|   |   |       |-- BrainMapping.jsx
|   |   |       |-- VitalsGrid.jsx
|   |   |       |-- NextSteps.jsx
|   |   |-- context/AuthContext.jsx
|   |   |-- lib/firebase.js, defaultData.js
|   |-- login/, signup/       # Auth pages
|   |-- functions/            # Firebase Cloud Functions
|   |-- firebase.json, firestore.rules
|   |-- package.json, vite.config.js
|
|-- Data/                     # Training data (MIMIC-IV)
|-- patient_risk_model/       # Earlier model experiments

```

-- Assets/

Design assets

12. Performance Benchmarks

ML Inference Performance

Metric	Min	Mean	P95	P99
Single Prediction	0.31ms	0.37ms	0.40ms	0.50ms

Batch Throughput: 550,448 predictions/second (10,000 batch)
Model Size: 2.8 MB
Model Load Time: ~300ms

Cross-Validation Stability

Fold	Accuracy	Fold	Accuracy
Fold 1	100.00%	Fold 6	99.98%
Fold 2	99.98%	Fold 7	99.98%
Fold 3	99.98%	Fold 8	99.98%
Fold 4	99.98%	Fold 9	100.00%
Fold 5	99.96%	Fold 10	100.00%

CV Mean: 99.98% +/- 0.01%
Consistency: Excellent (range: 99.96% - 100.00%)

Target Verification Summary

Metric	Target	Achieved	Status
Overall Accuracy	>= 98%	99.98%	PASS
High-Risk Recall	>= 95%	100.00%	PASS
Medium-Risk Recall	>= 90%	99.95%	PASS
Low-Risk Recall	>= 95%	100.00%	PASS
Macro F1-Score	>= 0.90	0.9999	PASS
Train/Test Gap	< 3%	0.02%	PASS
Inference (P95)	< 100ms	0.40ms	PASS

All 7/7 production targets met. Model approved for production deployment.

13. Future Roadmap

Planned Improvements

Wearable Integration

Connect with Apple Watch, Fitbit, and other wearable devices for continuous vital sign monitoring and automated assessment triggers.

Multi-Language Support

Extend the AI health companion to support Hindi, Spanish, and other languages for broader accessibility.

Push Notifications

Implement real-time push notifications for medication reminders, abnormal vital alerts, and appointment reminders.

Provider Portal

Build a separate healthcare provider dashboard for managing multiple patients, viewing population-level analytics, and receiving critical alerts.

FHIR Integration

Integrate with HL7 FHIR standards for interoperability with Electronic Health Record (EHR) systems.

Model Retraining Pipeline

Automate quarterly model retraining with drift detection, A/B testing of model versions, and automated rollback capabilities.

Explainable AI Dashboard

Add SHAP waterfall plots, LIME explanations, and counterfactual analysis for full model interpretability.

Offline Mode

Enable offline assessment capability using a lightweight on-device model for areas with limited connectivity.

End of Documentation

CORTEX v3.0 - Clinical Risk Assessment System
Comprehensive Project Documentation

Model: XGBoost | Accuracy: 99.98% | High-Risk Recall: 100%
Trained on 1,410,937 real patient records

Generated: February 2026